

❏ 1. Introduction to Logging & Monitoring

Logging and monitoring are **critical for system health**, troubleshooting, and security compliance.

- **Logging** captures what's happening (e.g., user login, service status).
- **Monitoring** observes resource usage and service availability.

❏ 2. What Can Be Monitored Using Bash?

Category	Examples
System Resources	CPU, Memory, Disk usage
Service Status	nginx, apache2, ssh, mysql
File Integrity	Changes in critical files
User Activity	Logins, sudo usage, cron jobs
Network	Interface status, open ports

☒ 3. Logging Basics in Bash

You can redirect output to a log file:

```
bash
```

Copy code

```
echo "[$(date)] Service started." >> /var/log/myscript.log
```

Or log both stdout and stderr:

```
bash
```

Copy code

```
./script.sh >> /var/log/script.log 2>&1
```

❏ 4. Monitoring System Metrics – Sample Script

bash

Copy code

```
#!/bin/bash logfile="/var/log/system_monitor.log" { echo "==== System Report: $(date) =====" echo "Disk Usage:" df -h echo "Memory Usage:" free -m echo "CPU Load:" uptime echo "Running Processes:" ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%mem | head echo "Network Interfaces:" ip a echo } >> "$logfile"
```

❏ ❏ 5. Monitoring Service Status

bash

Copy code

```
#!/bin/bash SERVICES=("ssh" "apache2" "mysql") LOGFILE="/var/log/service_status.log" for service in "${SERVICES[@]"; do systemctl is-active --quiet "$service" if [ $? -ne 0 ]; then echo "[$(date)] $service is DOWN" >> "$LOGFILE" else echo "[$(date)] $service is RUNNING" >> "$LOGFILE" fi done
```

✉ 6. Alerting by Email or Message

bash

Copy code

```
#!/bin/bash # Send email if disk usage > 90% THRESHOLD=90 USAGE=$(df / | grep / | awk '{ print $5 }' | sed 's/%//g') EMAIL="admin@example.com" if [ "$USAGE" -gt "$THRESHOLD" ]; then echo "Disk usage is above $THRESHOLD%" | mail -s "Disk Alert" "$EMAIL" fi
```

☒ 7. Real-World Case Study: “Web Server Monitoring”

☒ Scenario:

You manage a production **Apache web server** and want to automate:

- Service availability check
 - Disk and RAM usage
 - Log archiving
 - Weekly email summary
-

☒ Tools Used:

- Bash
- Cron
- mailx
- df, free, systemctl, tar, gzip

☒ Script: webserver_monitor.sh

bash

Copy code

```
#!/bin/bash LOG_DIR="/var/log/apache2" ARCHIVE_DIR="/root/weekly_logs" STATUS_FILE="/var/log/web_status.log"
EMAIL="admin@example.com" mkdir -p "$ARCHIVE_DIR" # Log server status { echo "=== Apache Status Report - $(date) ===" systemctl is-
active apache2 && echo "Apache is running." || echo "Apache is DOWN!" echo "Disk Usage:" df -h / echo "RAM Usage:" free -m } >>
"$STATUS_FILE" # Archive logs tar -czf "$ARCHIVE_DIR/apache_logs_$(date +%F).tar.gz" $LOG_DIR # Email summary mail -s "Weekly Apache
Report" "$EMAIL" < "$STATUS_FILE"
```

✂ Add to Cron (Weekly)

bash

Copy code

```
0 0 * * 0 /root/webserver_monitor.sh
```

✂ Best Practices

- Rotate logs to avoid size bloating (logrotate)
- Include timestamps in logs
- Use separate log files for errors and outputs
- Secure script permissions (chmod 700 script.sh)
- Automate alerting (SMS/email/Slack)